

Análisis Técnico

Sistema de Pagos — Prueba Técnica Backend

1. Cumplimiento de Requisitos

El proyecto cubre el **100% de los requisitos** del documento con detalles que van más allá del mínimo exigido.

Requisito	Estado
API REST Node.js — usuarios, tarjetas, pagos, historial	OK
PostgreSQL con 3 tablas, relaciones e índices	OK
Microservicio Python FastAPI (80% / 20% aleatorio)	OK
Node.js consume el servicio Python con timeout	OK
Colección Postman	OK
README con instrucciones claras	OK

Aspectos destacados que van más allá del mínimo:

- Hash SHA-256 del número de tarjeta; CVV nunca almacenado
- Fallback graceful cuando el microservicio Python no responde (pago queda *rejected* en lugar de crash)
- Validación de que la tarjeta pertenece al usuario antes de procesar el pago
- Error handler centralizado y graceful shutdown con SIGINT
- Índices en BD sobre columnas de filtrado frecuente

2. Bug Detectado (Crítico)

La ruta de historial de pagos está montada incorrectamente. El README documenta **GET /api/users/:id/payments** pero el código real expone **GET /api/payments/:id/payments**.

Causa raíz:

```
app.js          -> app.use('/api/payments', paymentRoutes)
paymentRoutes.js -> router.get('/:id/payments', ...) => /api/payments/:id/payments [INCORRECTO]
```

Fix: Mover el handler *getPayments* a *userRoutes.js* o ajustar el prefijo del router.

3. Mejoras Recomendadas

#	Problema	Solución	Impacto
1	SHA-256 sin sal — brute-forceable para tarjetas (solo 16 dígitos numéricos)	HMAC-SHA256 con CARD_SECRET desde variable de entorno	Seguridad
2	Tarjeta duplicada no detectada — mismo número se puede registrar varias veces	Verificar numero_hash + usuario_id antes de insertar (409 Conflict)	Integridad
3	Expiración solo valida formato MM/AAAA, no si la tarjeta ya venció	Comparar fecha contra el mes/año actual al registrar	Negocio
4	findCardById hace SELECT * exponiendo numero_hash al resto del código	Usar columnas explícitas que excluyan numero_hash	Seguridad
5	pool.on('error') llama a process.exit(-1) ante cualquier error del pool	Solo loguear; dejar que el pool se recupere automáticamente	Resiliencia
6	No existe GET /api/users/:id — no hay forma de consultar un usuario	Agregar endpoint de lectura de usuario (200 / 404)	Compleitud

4. Feature Extra: Endpoint de Estadísticas

Sin ser requerido en el documento, se propone agregar un endpoint de estadísticas de pagos por usuario. Su implementación es mínima (una sola query SQL con funciones de agregación) y aporta alto valor real: cualquier dashboard o reporte financiero lo necesitaría.

Ruta propuesta

```
GET /api/users/:id/payments/stats
```

Respuesta ejemplo

```
{
  "data": {
    "total_pagos": 10,
    "aprobados": 8,
    "rechazados": 2,
    "monto_total_aprobado": 12500.00,
    "monto_promedio": 1250.00
  }
}
```

Query SQL requerida

```
SELECT
  COUNT(*) AS total_pagos,
  COUNT(*) FILTER (WHERE estado = 'approved') AS aprobados,
  COUNT(*) FILTER (WHERE estado = 'rejected') AS rechazados,
```

```
COALESCE(SUM(monto) FILTER (WHERE estado = 'approved'), 0) AS monto_total_aprobado,  
COALESCE(AVG(monto), 0) AS monto_promedio  
FROM pagos WHERE usuario_id = $1;
```

Por qué aporta valor

- Demuestra pensamiento de producto, no solo implementación técnica
- Cero dependencias nuevas — solo SQL nativo de PostgreSQL
- Aprovecha los índices ya existentes (idx_pagos_usuario, idx_pagos_estado)
- Es la base de cualquier dashboard de administración o reporte financiero